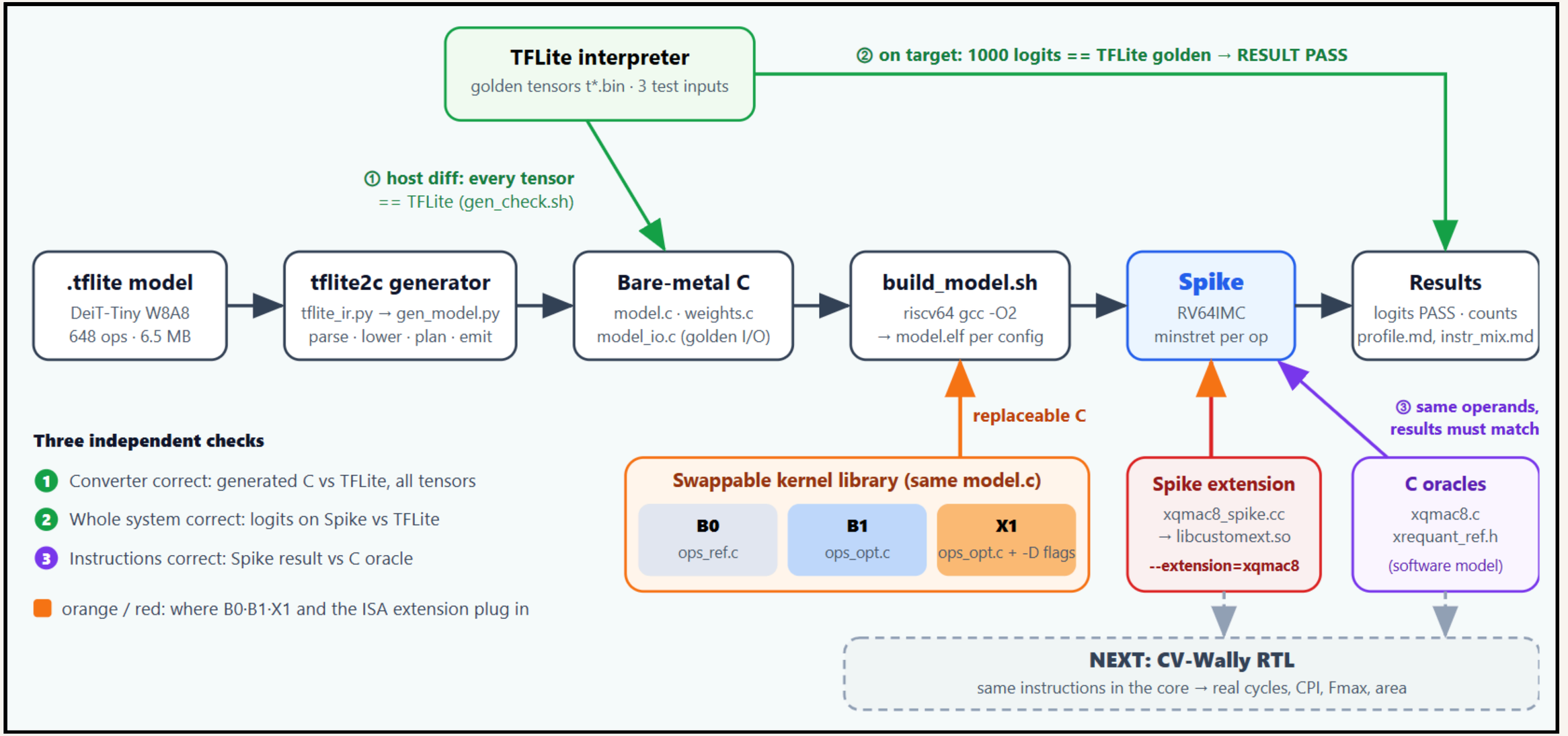
Computer Architecture Optimization for DeiT-Tiny Transformer Model

Midterm checkpoint: Software and RISC-V Instruction Optimizations

---

Course: Advanced Computer Architecture
Name: Rachel Yang, X1150022

Implementation



Model Details & Optimization Plan

The model: DeiT-Tiny

5.7M parameters
1.25GMACS/image
648 graph ops; 16 kinds
5.8MB of weights +biases
W8A8

B0: baseline

Generated from tflite to c scripts

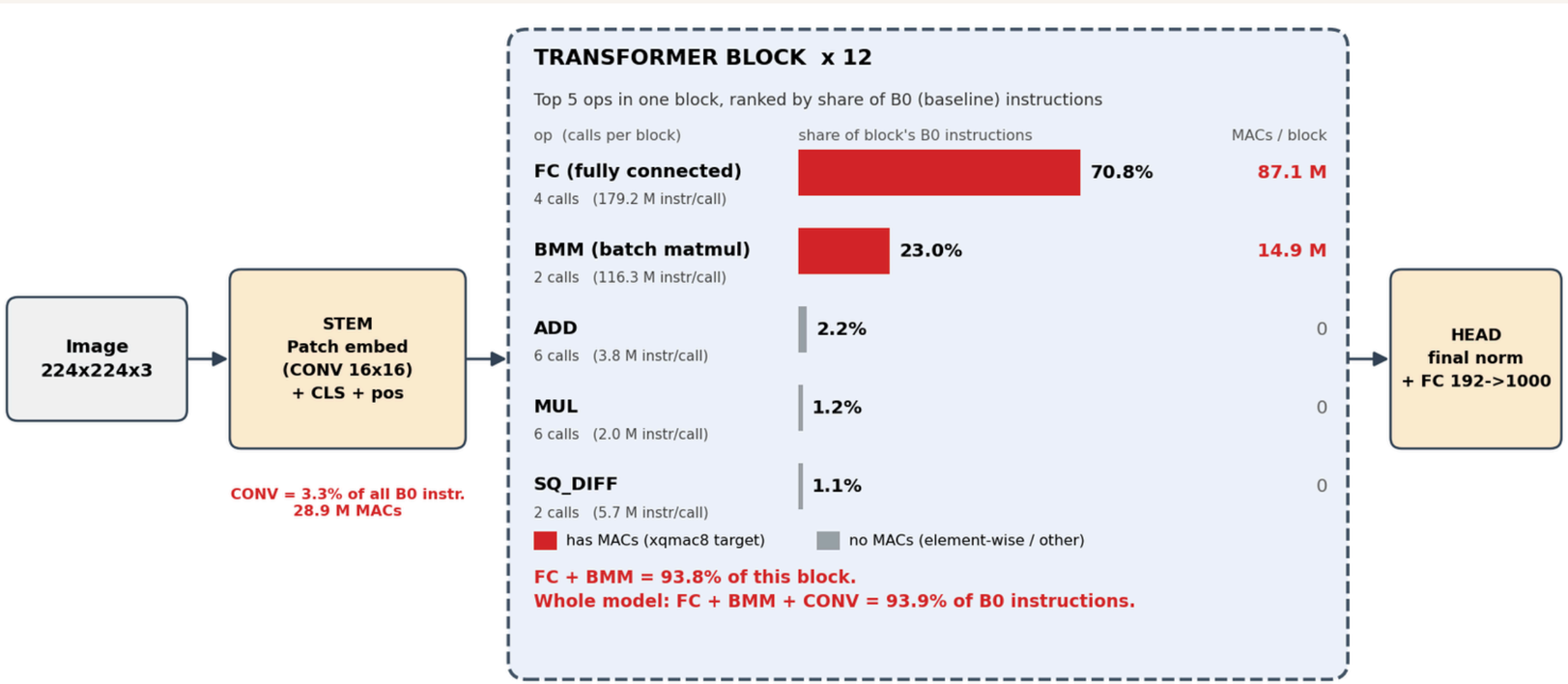
(x – zero_point) × w for every MAC loop

B1

Software optimizations:

- zero_point moved out of MAC loop
- register blocking

Code reformat to ensure B1 to X1 software is identical for better baselining



X1

Implemented xqmac8 + xrequant

Why xqmac optimization?

- FC and BMM are the top portion of DeiT-Tiny RISC-V instructions
- Therefore, the three ops that use MACs (i.e. FC, BMM, andd CONV) make up 94% of instructions
- There are 1.25MACs
- Each MAC consists of:
  - 2 loads, ~5 ALU, 1 multiply, 1 branch

X1MAC

Same as X1 but without xrequant to differentiate xqmac8 vs xrequant optimization

Implementation Details of xqmac and xrequant

xqmac8

```
for (int32_t i = 0; i < k; i++) {
    int32_t v0 = x0[i], v1 = y1[i];
    int32_t u0 = w0[i], u1 = w1[i], u2 = w2[i], u3 = w3[i];
    s00 += v0 * u0; s01 += v0 * u1; s02 += v0 * u2; s03 += v0 * u3;
    s10 += v1 * u0; s11 += v1 * u1; s12 += v1 * u2; s13 += v1 * u3;
}
```

for gemm_nt_rows() ↓ const int32_t kw = k / 8;

```
for (int32_t i = 0; i < kw; i++) {
    uint64_t v0 = a0[i], v1 = a1[i];
    uint64_t u0 = w0[i], u1 = w1[i], u2 = w2[i], u3 = w3[i];
    s00 = xqmac8(s00, v0, u0); s01 = xqmac8(s01, v0, u1);
    s02 = xqmac8(s02, v0, u2); s03 = xqmac8(s03, v0, u3);
    s10 = xqmac8(s10, v1, u0); s11 = xqmac8(s11, v1, u1);
    s12 = xqmac8(s12, v1, u2); s13 = xqmac8(s13, v1, u3);
}
```

How xqmac8 interfaces with the spike instruction:

```
#ifdef GEMM_XQMAC8
static inline int64_t xqmac8(int64_t acc, uint64_t a, uint64_t b)
{
    __asm__(".insn r 0x0b, 0, 0x2b, %0, %1, %2" : "+r"(acc) : "r"(a), "r"(b));
    return acc;
}
#endif
```

xrequant

```
static inline int32_t tfl_mul_qm(int32_t x, int32_t multiplier, int shift)
{
    int left = shift > 0 ? shift : 0;
    int right = shift > 0 ? 0 : -shift;
    return tfl_rdivpot(tfl_srdhm((int32_t)((uint32_t)x << left), multiplier), right);
}
```

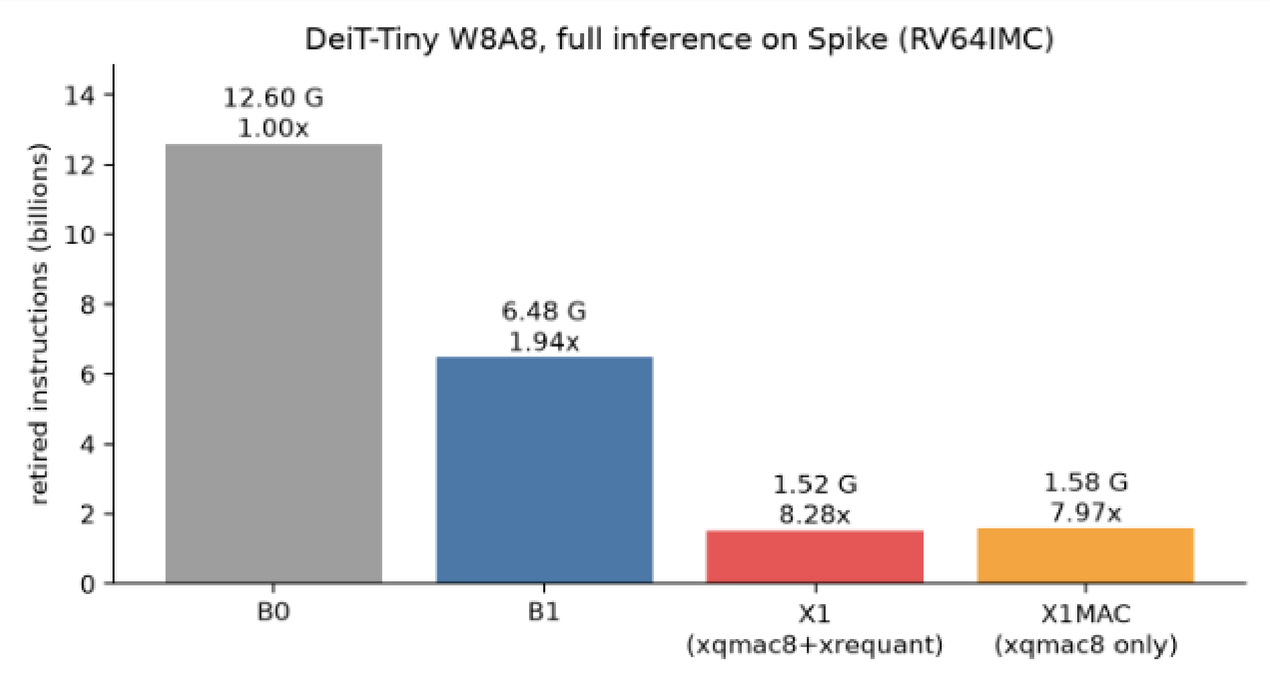
↓

```
static inline int32_t xrequant(int32_t acc, int32_t mult, int32_t shift, int32_t zp, int mode)
{
    int left = shift > 0 ? shift : 0;
    int right = shift > 0 ? 0 : -shift;
    acc = (int32_t)((uint32_t)acc << left);
    int64_t r = (int64_t)(((uint64_t)mode << 16) | ((uint64_t)right << 8) | ((uint64_t)zp & 0xff));
    __asm__(".insn r 0x0b, 1, 0x2b, %0, %1, %2" : "+r"(r) : "r"((int64_t)acc), "r"((int64_t)mult));
    return (int32_t)r;
}
```

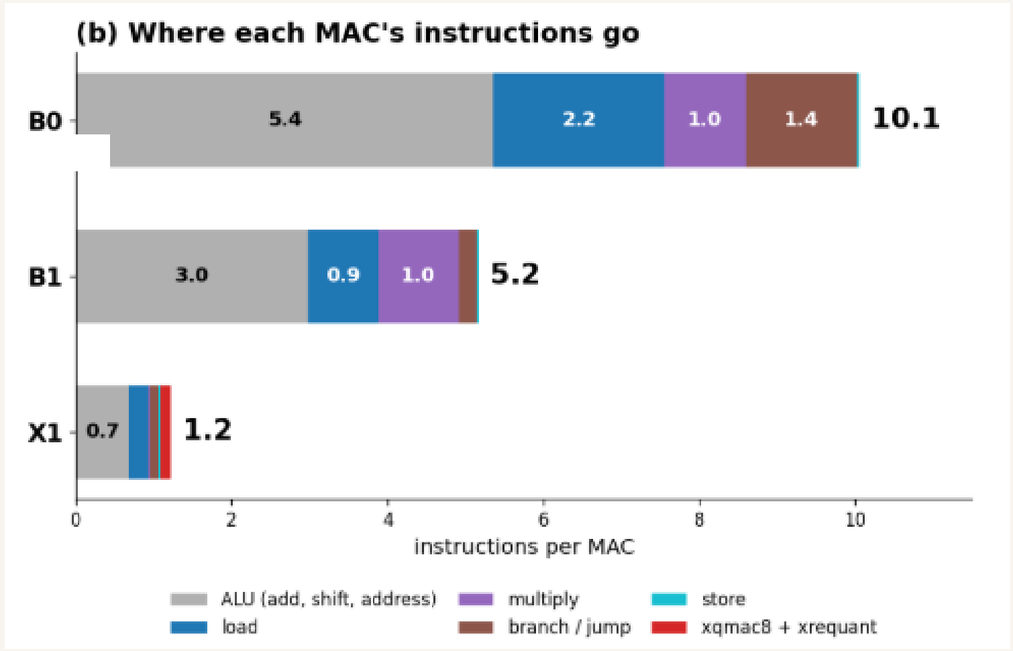
xrequant replaces tfl_mut_qm c implementation into hardware:

```
#ifdef USE_XREQUANT
#define RQ_UP(acc, mult, shift, zp, lo, hi) tfl_clamp_i8(xrequant((acc), (mult), (shift), (zp), 1), (lo), (hi))
#define RQ_GEMM(acc, mult, shift, zp, lo, hi) tfl_clamp_i8(xrequant((acc), (mult), (shift), (zp), 2), (lo), (hi))
#else
#define RQ_UP(acc, mult, shift, zp, lo, hi) tfl_clamp_i8(tfl_mul_qm_up((acc), (mult), (shift)) + (zp), (lo), (hi))
#define RQ_GEMM(acc, mult, shift, zp, lo, hi) tfl_clamp_i8(tfl_mul_qm((acc), (mult), (shift)) + (zp), (lo), (hi))
#endif
```

Results



	B0	B1	X1
Instructions / MAC	10.05	5.17	**1.22**
.text code size (bytes)	31,786	34,192	34,214



Reference Matching: Every config confirmed 1000/1000 class scores match tflite

Project Requirements Target:
≥2× reduction in dynamic instructions for a major GEMM/FC kernel
Achieved:
5.39G in B1 → 0.49G instr, which is 11x reduction

Thank you!